

probib.sty

A problem library with usage tracking for the `exam` class

Kyle Monette (kylemonette.github.io)

Released under the LaTeX Project Public License (LPPL), version 1.3c

October 4, 2026 — Version 1.0

Overview

`probib` keeps a library of individual problems, each stored in its own file with metadata such as topics, tags and difficulty. Assignments are `exam`-class documents that pull problems from the library by path. Settings that belong to the assignment rather than the problem, such as the blank space left for students' work, are made in the assignment itself, so the same problem can be reused anywhere.

Every assignment records which problems it used. A catalog document, compiled with Lua $\text{\LaTeX}$, gathers those records into a usage history and typesets the library with each problem's metadata and the assignments it appeared in. Assignments can show that history next to each problem and warn when a problem was used recently.

A typical workflow is:

1. write problems into the library (Section 3);
2. build assignments from them with `\useproblem` (Section 4);
3. compile the catalog to update the usage history (Section 7).

Complete examples are in the `probib` repository at <https://github.com/kylemonette/probib>.

Contents

1 Quick reference	2
2 Requirements	2
3 Problem files	2
3.1 Metadata	3
4 Building an assignment	3
4.1 Settings	3
4.2 Describing the assignment	4
4.3 Inserting problems	4
5 Answer space	4
5.1 Answer keys	6
6 Usage tracking	6
6.1 Showing usage in an assignment	7
6.2 Reuse warnings	7

7	The catalog	7
8	Implementation	8
8.1	Messages and dependencies	8
8.2	Settings	9
8.3	The assignment record	10
8.4	Dates	11
8.5	The usage database	12
8.6	Answer space	13
8.7	Problems	15
8.8	Notes and tables	16
8.9	The catalog	18
8.10	Options and mode	19
8.11	Lua code	20

1 Quick reference

Problem files	
<code>\begin{problem}[metadata]</code>	Holds one problem in a library file; its metadata goes in the optional argument.
<code>\problemmeta{⟨key⟩}</code>	Expands to a metadata value of the problem being typeset.
Assignments	
<code>\usepackage[⟨options⟩]{probib}</code>	Loads the package; options are the keys of <code>\probibsetup</code> , usually library .
<code>\probibassignment{⟨keys⟩}</code>	Preamble only. Records the assignment's name, course, term, type and date.
<code>\probibsetup{⟨keys⟩}</code>	Sets defaults: library folder, showdata , warnreuse , ...
<code>\useproblem[⟨options⟩]{⟨path⟩}</code>	Typesets the problem at <code>⟨path⟩</code> and records its use.
<code>\problemspace[⟨style⟩]{⟨length⟩}</code>	Adds answer space at that point.
Catalog (using Lua ^L A ^T E ^X)	
<code>\usepackage[catalog]{probib}</code>	Loads the package in catalog mode; the roots option lists the folders holding assignments.
<code>\probibcatalog[⟨filters⟩]</code>	Updates the usage database and typesets the matching problems with their metadata and usage.

2 Requirements

`probib` requires the `exam` document class and a ^LA^TE^X kernel from November 2023 or later; it loads `xcolor` and `tabularx`. Assignments compile with pdf^LA^TE^X, Xe^LA^TE^X or Lua^LA^TE^X. The catalog requires Lua^LA^TE^X, which it uses to list the contents of folders.

3 Problem files

A library is a folder of problem files, organized into subfolders however you like. Each file holds one **problem** environment and nothing else. A problem is referred to by its path inside the library, without the `.tex` extension.

```

\begin{problem}[
  title = Chain rule with trig,
  topics = {calculus, derivatives},
  tags = {chain rule, trig},
  difficulty = 2,
  time = 5,
  author = John Smith,
  created = 2025-08-14,
]
Differentiate each function.
\begin{parts}
  \part[4]  $f(x) = \sin(x^2)$ 
  \part[6]  $g(x) = \cos^3(2x)$ 
\end{parts}
\begin{solution}
  (a)  $2x\cos(x^2)$  \quad (b)  $-6\cos^2(2x)\sin(2x)$ 
\end{solution}
\end{problem}

```

The body is written exactly as inside an `exam` `\question`: use `parts`, `subparts`, `\part[⟨points⟩]`, `solution` and the other environments of the class.

3.1 Metadata

The optional argument of `problem` is a list of $\langle key \rangle = \langle value \rangle$ pairs. Any key may be used, and every key is shown in the catalog in the order written. A few keys have a special meaning:

<code>title</code>	Shown next to the problem's path in the catalog and in <code>showdata</code> notes.
<code>points</code>	The question's point value, passed to <code>\question[⟨points⟩]</code> .
<code>topics, tags</code>	Comma lists, matched by the catalog filters of the same names.
<code>difficulty</code>	An integer, used by the catalog to filter and sort.

Brace a value that contains a comma or a closing bracket, as in `tags = {chain rule, trig}`. Give points either with `points` or on the individual parts, not both: `exam` adds them together.

Inside the problem, `\problemmeta{⟨key⟩}` expands to the value of $\langle key \rangle$, or to nothing if the key is not set; for example, `\problemmeta{time}` gives the estimated time of the problem above.

4 Building an assignment

An assignment is an `exam` document that loads `problib`, describes itself with `\problibassignment`, and calls `\useproblem` inside a `questions` environment. Its own `\questions` can be mixed freely with library problems.

```

\documentclass[addpoints]{exam}
\usepackage{amsmath}
\usepackage[library = /Users/me/problems]{problib}

\problibassignment{
  name = Exam 1,
  course = MATH 221,
  term = Fall 2026,
  type = exam,
  date = 2026-10-12,
}
\problibsetup{problemspace = \fill, partspace = 1.5in, warnreuse = 120}

```

```

\begin{document}
\begin{questions}
\useproblem{calculus/derivatives/chain-rule-01}
\useproblem[points = 10]{calculus/integrals/u-sub-01}
\useproblem[newpage]{calculus/limits/squeeze-01}
\end{questions}
\end{document}

```

4.1 Settings

`\problibsetup{<keys>}` sets the defaults for the whole document. The same keys may be given as package options.

library	The folder holding the problems, absolute or relative to the assignment's folder. Without it, problem paths are taken relative to the assignment's folder.
problemspace, ...	The answer-space keys problemspace , partspace , subpartspace , spacestyle and keyspace ; see Section 5.
showdata	Show each problem's usage history above it; see Section 6.1.
warnreuse	Warn about recently used problems; see Section 6.2.
roots	Catalog only; see Section 7.

4.2 Describing the assignment

`\problibassignment{<keys>}` may only be used in the preamble. Its keys are written to the assignment's usage record (Section 6) and identify the assignment in the usage history.

name	Name of the assignment. Default: the file name.
course, term, type	Free text, shown in the usage history.
date	YYYY-MM-DD. Default: the day of compilation. Orders the history and is the reference date for warnreuse .
record	false keeps the assignment out of the usage history (drafts, practice versions). Default: true .

4.3 Inserting problems

`\useproblem[<options>]{<path>}` reads the file `<library>/<path>.tex`, adds `<path>` to the assignment's usage record, and typesets the problem as the next `\question`, with the point value from its **points** metadata if it has one. A missing file is an error and is not recorded. The options apply to this problem only:

space, partspace, subpartspace, spacestyle	Answer space; see Section 5. space sets problemspace .
points	Points for the question, overriding the problem's points .
newpage	Start the question on a new page.

```
\begin{questions}
\useproblem{demo-derivatives}
\useproblem[points = 8]{demo-integral}
\end{questions}
```

1. Differentiate each function.
 - (a) (3 points) $f(x) = \sin(x^2)$
 - (b) (3 points) $g(x) = x^2 e^x$
 - (c) (4 points) $h(x) = \ln(\cos x)$
2. (8 points) Evaluate $\int x\sqrt{x^2+1} dx$.

5 Answer space

Answer space is chosen by the assignment and never stored in a problem. Defaults are set with `\problibsetup` and overridden for a single problem with the options of `\useproblem`. By default no space is added.

problemspace	Space after a question that has no parts.
partspace	Space after each part. A single length applies to every part; a comma list gives the space after the first, second, ... part, and its last value is reused for any further parts.
subpartspace	The same, for the subparts of each part.
spacestyle	How the space is filled: blank (default), lines , dottedlines , box or grid .
keyspace	Keep the space in answer keys (when <code>\printanswers</code> is on). Default: false .

Lengths are anything `exam` accepts, including `\fill` and `\stretch{\langle n \rangle}`. The styles other than **blank** use `exam`'s `\fillwithlines`, `\fillwithdottedlines`, `\makeemptybox` and `\fillwithgrid`.

Space goes only where answers are written, at the innermost level of a problem. A question with parts gets **partspace** after each part and no **problemspace**; a part with subparts gets **subpartspace** after each subpart and no **partspace**.

Here each part gets its own space from a list:

```
\begin{questions}
\useproblem[partspace = {0.2in, 0.6in, 0.3in}]{demo-derivatives}
\end{questions}
```

1. Differentiate each function.
 - (a) (3 points) $f(x) = \sin(x^2)$
 - (b) (3 points) $g(x) = x^2 e^x$
 - (c) (4 points) $h(x) = \ln(\cos x)$

Here part (a) has subparts, so the space follows each subpart, while part (b) gets **partspace**:

```
\begin{questions}
\useproblem[partspace = 0.4in, subpartspace = 0.2in]{demo-eigen}
\end{questions}
```

1. Let $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$.
 - (a) Find the eigenpairs of A .
 - i. (3 points) Eigenvalues.
 - ii. (3 points) Eigenvectors.
 - (b) (4 points) Diagonalize A .

Questions without parts use `space`, shown here in two styles:

```
\begin{questions}
\useproblem[space = 0.8in, spacestyle = lines]{demo-integral}
\useproblem[space = 0.8in, spacestyle = box]{demo-integral}
\end{questions}
```

1. (5 points) Evaluate $\int x\sqrt{x^2+1} dx$.

2. (5 points) Evaluate $\int x\sqrt{x^2+1} dx$.

`\problemspace[<style>]{<length>}` adds space at the point where it is used, for example after a `\question` of your own. It uses the current `spacestyle` unless *<style>* is given.

5.1 Answer keys

`\printanswers` turns the same assignment into its answer key: solutions appear and answer space is dropped. Set `keyspace = true` to keep the space, so that the key has the same page layout as the assignment.

```
\printanswers
\begin{questions}
\useproblem[partspace = 0.5in]{demo-derivatives}
\end{questions}
```

1. Differentiate each function.

(a) (3 points) $f(x) = \sin(x^2)$

(b) (3 points) $g(x) = x^2 e^x$

(c) (4 points) $h(x) = \ln(\cos x)$

Solution: (a) $2x \cos(x^2)$ (b) $(2x + x^2)e^x$ (c) $-\tan x$

6 Usage tracking

TeX may write files only in the folder of the document being compiled, so each assignment keeps its own record. Compiling an assignment writes $\langle jobname \rangle.pbu$ next to it, where $\langle jobname \rangle$ is the file name of the assignment. The record lists the details from `\probibassignment` and every problem inserted with `\useproblem`. It is rewritten on every compilation, so it always matches the current assignment, and it never needs editing. Deleting it removes the assignment from the history; compiling the assignment again restores it.

Compiling the catalog (Section 7) collects every record under the `roots` folders into the usage database, `usage-db.tex` in the library. Assignments read that file, so the history they see is as current as the last catalog compilation. Until the catalog has been compiled once, assignments compile normally and `probib` issues a warning that the database is missing.

6.1 Showing usage in an assignment

With `showdata = true`, every problem is preceded by its path, its title, and a table of its earlier uses, newest first.

```
\probibsetup{showdata = true}
\begin{questions}
\useproblem{demo-derivatives}
\useproblem{demo-integral}
\end{questions}
```

1. demo-derivatives *Derivatives*

date	course	term	type	assignment
2026-02-14	MATH 221	Spring 2026	quiz	Quiz 2
2025-09-30	MATH 221	Fall 2025	worksheet	Worksheet 4

Differentiate each function.

(a) (3 points) $f(x) = \sin(x^2)$

(b) (3 points) $g(x) = x^2 e^x$

(c) (4 points) $h(x) = \ln(\cos x)$

2. (5 points) demo-integral *Substitution*

Not used before.

Evaluate $\int x\sqrt{x^2 + 1} dx$.

6.2 Reuse warnings

`warnreuse =  $\langle days \rangle$` checks each problem's most recent use in another assignment. If that use is less than $\langle days \rangle$ days before the assignment's `date` (or after it), `probib` writes a warning to the log, for example

```
Package probib Warning: Problem 'demo-derivatives' was used in
(probib)                'Quiz 2 (Spring 2026)' on 2026-02-14.
```

The default, 0, disables the check.

7 The catalog

The catalog is a Lua $\text{\LaTeX}$ document that lives in the library folder and must be compiled there: it treats the folder it is compiled in as the library.

```
\documentclass{exam}
\usepackage{amsmath}
\usepackage[catalog, roots = {~/Teaching}]{probib}
\begin{document}
\probibcatalog
\end{document}
```

Each compilation

1. finds every problem file in the library, including subfolders,
2. reads every `.pbu` record under the `roots` folders, including subfolders,
3. writes the usage database `usage-db.tex` into the library, and
4. typesets the matching problems, each headed by its path and title, with a table of its metadata and a table of every assignment that used it.

`roots` is a comma list of folders, absolute or relative to the library; a leading `~` stands for the home folder. Records of assignments with `record = false` are ignored.

`\probibcatalog[ $\langle filters \rangle$ ]` with no filters lists every problem. It may be used several times in one catalog, for example once per section with a different `folders` filter. The filters are:

<code>folders</code>	Only problems under these library folders.
<code>tags, topics</code>	Only problems with at least one of the listed values (case-insensitive).
<code>difficulty</code>	Only problems with this difficulty (2) or a difficulty in this range (1-3).
<code>unusedsince</code>	Only problems not used on or after this date (YYYY-MM-DD).
<code>sort</code>	Order: <code>path</code> (default), <code>difficulty</code> , or <code>lastused</code> , which lists never-used and least recently used problems first.
<code>show</code>	What to show with each problem: any of <code>metadata</code> , <code>usage</code> and <code>solution</code> . Default: <code>{metadata, usage}</code> .

For instance, to find trigonometry problems in the calculus folder that were not used during the last year, least recently used first:

```
\probibcatalog[folders = calculus, tags = {trig}, unusedsince = 2025-10-01, sort = lastused]
```

The example catalog in the repository's `library` folder shows the result for a small library.

8 Implementation

8.1 Messages and dependencies

```
<*package>
\msg_new:nnn { probleb } { no-exam }
{ The~probleb~package~requires~the~exam~document~class. }
\msg_new:nnn { probleb } { no-luatex }
{ Catalog~mode~requires~LuaLaTeX. }
\msg_new:nnn { probleb } { not-catalog }
{ \token_to_str:N \problebcatalog \ requires~the~'catalog'~package~option. }
\msg_new:nnn { probleb } { missing }
{ Problem~file~'#1.tex'~not~found. }
\msg_new:nnn { probleb } { no-db }
{
  No~usage~database~at~'#1'.~Compile~the~library~catalog~with~LuaLaTeX~
  to~create~it.
}
\msg_new:nnn { probleb } { bad-date }
{ Assignment~date~'#1'~is~not~YYYY-MM-DD;~reuse~warnings~disabled. }
\msg_new:nnn { probleb } { reuse }
{ Problem~'#1'~was~used~in~'#2'~on~#3. }
\msg_new:nnn { probleb } { bad-show }
{ Unknown~catalog~'show'~item~'#1'. }

\IfClassLoadedTF { exam } { } { \msg_error:nn { probleb } { no-exam } }
\RequirePackage { xcolor }
\RequirePackage { tabularx }
```

8.2 Settings

Keys of `\problebsetup`, which are also the package options.

```
\str_new:N \l__probleb_library_str
\str_new:N \l__probleb_roots_str
\tl_new:N \l__probleb_problemspace_tl
\clist_new:N \l__probleb_partspace_clist
\clist_new:N \l__probleb_subpartspace_clist
\str_new:N \l__probleb_spacestyle_str
\bool_new:N \l__probleb_showdata_bool
\bool_new:N \l__probleb_keyspace_bool
\int_new:N \l__probleb_warnreuse_int
\bool_new:N \g__probleb_catalog_bool

\keys_define:nn { probleb }
{
  library .code:n = \__probleb_set_library:n {#1} ,
  roots .code:n = \str_set:Nn \l__probleb_roots_str {#1} ,
  problemspace .tl_set:N = \l__probleb_problemspace_tl ,
  partspace .clist_set:N = \l__probleb_partspace_clist ,
  subpartspace .clist_set:N = \l__probleb_subpartspace_clist ,
  spacestyle .choices:nn = { blank, lines, dottedlines, box, grid }
  { \str_set:NV \l__probleb_spacestyle_str \l_keys_choice_tl } ,
  spacestyle .initial:n = blank ,
  showdata .bool_set:N = \l__probleb_showdata_bool ,
  keyspace .bool_set:N = \l__probleb_keyspace_bool ,
  warnreuse .int_set:N = \l__probleb_warnreuse_int ,
  catalog .bool_gset:N = \g__probleb_catalog_bool ,
```

```

    catalog      .usage:n      = load ,
  }

\cs_new_protected:Npn \__problib_set_library:n #1
{
  \str_set:Nn \l__problib_library_str {#1}
  \tl_trim_spaces:N \l__problib_library_str
  \str_if_empty:NF \l__problib_library_str
  {
    \str_if_eq:eeF { \str_item:Nn \l__problib_library_str { -1 } } { / }
    { \str_put_right:Nn \l__problib_library_str { / } }
  }
}

\NewDocumentCommand \problibsetup { m } { \keys_set:nn { problib } {#1} }

```

Options of `\useproblem`, which forward the spacing keys to the main family inside the group of the command.

```

\tl_new:N \l__problib_points_tl
\bool_new:N \l__problib_newpage_bool
\bool_new:N \l__problib_active_bool

\keys_define:nn { problib / use }
{
  space      .meta:nn      = { problib } { problemspace = {#1} } ,
  partspace  .meta:nn      = { problib } { partspace = {#1} } ,
  subpartspace .meta:nn    = { problib } { subpartspace = {#1} } ,
  spacestyle .meta:nn      = { problib } { spacestyle = {#1} } ,
  points      .tl_set:N     = \l__problib_points_tl ,
  newpage     .bool_set:N   = \l__problib_newpage_bool ,
  newpage     .default:n    = true ,
}

```

8.3 The assignment record

At the start of the document the assignment's details receive their defaults, the record file `\jobname\pbu` is opened and its header written, and the usage database is loaded.

```

\prop_new:N \g__problib_assign_prop
\bool_new:N \g__problib_record_bool
\str_new:N \g__problib_current_str
\int_new:N \g__problib_today_int
\iow_new:N \g__problib_iow
\bool_gset_true:N \g__problib_record_bool

\keys_define:nn { problib / assignment }
{
  name      .prop_gput:N = \g__problib_assign_prop ,
  course    .prop_gput:N = \g__problib_assign_prop ,
  term      .prop_gput:N = \g__problib_assign_prop ,
  type      .prop_gput:N = \g__problib_assign_prop ,
  date      .prop_gput:N = \g__problib_assign_prop ,
  record    .bool_gset:N = \g__problib_record_bool ,
}

\NewDocumentCommand \problibassignment { m }

```

```

    { \keys_set:nn { probleb / assignment } {#1} }
\@onlypreamble \problebassignment

\cs_new:Npn \__probleb_two:n #1
  { \int_compare:nNnT {#1} < { 10 } { 0 } \int_eval:n {#1} }

\cs_new:Npn \__probleb_field:n #1
  { #1 = { \prop_item:Nn \g__probleb_assign_prop {#1} } , }

\cs_new_protected:Npn \__probleb_begin_assignment:
  {
    \prop_if_in:NnF \g__probleb_assign_prop { name }
      { \prop_gput:NnV \g__probleb_assign_prop { name } \c_sys_jobname_str }
    \prop_if_in:NnF \g__probleb_assign_prop { date }
      {
        \prop_gput:Nne \g__probleb_assign_prop { date }
        {
          \int_use:N \c_sys_year_int -
          \__probleb_two:n { \c_sys_month_int } -
          \__probleb_two:n { \c_sys_day_int }
        }
      }
  }
\str_gset:Ne \g__probleb_current_str
  {
    \prop_item:Nn \g__probleb_assign_prop { name } |
    \prop_item:Nn \g__probleb_assign_prop { course } |
    \prop_item:Nn \g__probleb_assign_prop { term }
  }
\__probleb_set_today:e { \prop_item:Nn \g__probleb_assign_prop { date } }
\iow_open:Nn \g__probleb_iow { \c_sys_jobname_str .pbu }
\iow_now:Ne \g__probleb_iow
  {
    \token_to_str:N \problebrecord
    {
      \__probleb_field:n { name }
      \__probleb_field:n { course }
      \__probleb_field:n { term }
      \__probleb_field:n { type }
      \__probleb_field:n { date }
      record = { \bool_if:NTF \g__probleb_record_bool { true } { false } }
    }
  }
\__probleb_load_db:e { \l__probleb_library_str usage-db.tex }
}

```

8.4 Dates

Dates are converted to a day count (a Julian day number) so that reuse windows can be measured in days.

```

\regex_const:Nn \c__probleb_date_regex
  { \A \d{4} - (0?[1-9]|1[0-2]) - (0?[1-9]|1[12]\d|3[01]) \Z }

\cs_new:Npn \__probleb_days:n #1 { \__probleb_days:w #1 \q_stop }
\cs_new:Npn \__probleb_days:w #1 - #2 - #3 \q_stop
  {
    \__probleb_days:ffn
  }

```

```

    { \int_eval:n { #1 + 4800 - \int_div_truncate:nn { 14 - #2 } { 12 } } }
    { \int_eval:n { #2 + 12 * \int_div_truncate:nn { 14 - #2 } { 12 } - 3 } }
    {#3}
  }
\cs_new:Npn \__probib_days:nnn #1#2#3
{
  \int_eval:n
  {
    #3 + \int_div_truncate:nn { 153 * #2 + 2 } { 5 } + 365 * #1
    + \int_div_truncate:nn {#1} { 4 }
    - \int_div_truncate:nn {#1} { 100 }
    + \int_div_truncate:nn {#1} { 400 }
  }
}
\cs_generate_variant:Nn \__probib_days:nnn { ff }

\cs_new_protected:Npn \__probib_set_today:n #1
{
  \regex_match:NnTF \c__probib_date_regex {#1}
  { \int_gset:Nn \g__probib_today_int { \__probib_days:n {#1} } }
  {
    \msg_warning:nnn { problib } { bad-date } {#1}
    \int_set_eq:NN \l__probib_warnreuse_int \c_zero_int
  }
}
\cs_generate_variant:Nn \__probib_set_today:n { e }

```

8.5 The usage database

`usage-db.tex` holds one line per problem, `\probibusage{⟨path⟩}{⟨entries⟩}`, where each entry is `\probibentry{⟨date⟩}{⟨name⟩}{⟨course⟩}{⟨term⟩}{⟨type⟩}{⟨file⟩}`, newest first. It is read with the special characters of file paths made harmless, and each problem's entries are stored unexpanded so that `\probibentry` can be given different meanings when they are used.

```

\prop_new:N \g__probib_usage_prop
\bool_new:N \g__probib_db_bool

\cs_new_protected:Npn \__probib_load_db:n #1
{
  \file_if_exist:nTF {#1}
  {
    \group_begin:
    \clist_map_inline:nn { \#, \$, %, &, ^, \_, ~ }
    { \char_set_catcode_other:N ##1 }
    \cs_set_protected:Npn \probibusage ##1##2
    { \prop_gput:Nnn \g__probib_usage_prop {##1} {##2} }
    \file_input:n {#1}
    \group_end:
    \bool_gset_true:N \g__probib_db_bool
  }
  { \msg_warning:nnn { problib } { no-db } {#1} }
}
\cs_generate_variant:Nn \__probib_load_db:n { e }

```

For the current problem, fetch its entries and find the most recent one that does not belong to the assignment being compiled.

```

\str_new:N \l__probib_path_str
\tl_new:N \l__probib_uses_tl
\tl_new:N \l__probib_last_tl

\cs_new_protected:Npn \__probib_scan_usage:
{
  \tl_clear:N \l__probib_last_tl
  \prop_get:NVNF \g__probib_usage_prop \l__probib_path_str \l__probib_uses_tl
  { \tl_clear:N \l__probib_uses_tl }
  \cs_set_protected:Npn \probibentry ##1##2##3##4##5##6
  {
    \tl_if_empty:NT \l__probib_last_tl
    {
      \str_if_eq:eeF { \tl_to_str:n { ##2 | ##3 | ##4 } } { \g__probib_current_str }
      { \tl_set:Nn \l__probib_last_tl { {##1} {##2} {##3} {##4} } }
    }
  }
  \l__probib_uses_tl
}

\cs_new_protected:Npn \__probib_check_reuse:
{
  \bool_lazy_and:nnT
  { \int_compare_p:nNn \l__probib_warnreuse_int > \c_zero_int }
  { ! \tl_if_empty_p:N \l__probib_last_tl }
  { \exp_after:wN \__probib_check_reuse:nnnn \l__probib_last_tl }
}

\cs_new_protected:Npn \__probib_check_reuse:nnnn #1#2#3#4
{
  \regex_match:NnT \c__probib_date_regex {#1}
  {
    \int_compare:nNnT
    { \g__probib_today_int - \__probib_days:n {#1} } < \l__probib_warnreuse_int
    {
      \msg_warning:nneee { probib } { reuse } { \l__probib_path_str }
      { \exp_not:n {#2} \tl_if_blank:nF {#4} { ~( \exp_not:n {#4} ) } } {#1}
    }
  }
}

```

8.6 Answer space

A space is skipped when it is blank, zero, or when an answer key is being printed without `keyspace`. Values starting with a control sequence (`\fill`, `\stretch`) are always used.

```

\cs_new_protected:Npn \__probib_space:n #1
{
  \bool_lazy_or:nnF
  { \tl_if_blank_p:n {#1} }
  {
    \bool_lazy_and_p:nn
    { \legacy_if_p:n { printanswers } } { ! \l__probib_keyspace_bool }
  }
  {
    \tl_if_head_eq_catcode:nNTF {#1} \scan_stop:
    { \__probib_fill:n {#1} }
    { \dim_compare:nNnF {#1} = \c_zero_dim { \__probib_fill:n {#1} } }
  }
}

```

```

    }
  }
\cs_generate_variant:Nn \__problib_space:n { V, e }

\cs_new_protected:Npn \__problib_fill:n #1
  { \par \use:c { __problib_fill_ \l__problib_spacestyle_str :n } {#1} }
\cs_new_protected:Npn \__problib_fill_blank:n { \vspace }
\cs_new_protected:Npn \__problib_fill_lines:n { \fillwithlines }
\cs_new_protected:Npn \__problib_fill_dottedlines:n { \fillwithdottedlines }
\cs_new_protected:Npn \__problib_fill_box:n { \makeemptybox }
\cs_new_protected:Npn \__problib_fill_grid:n { \fillwithgrid }

\NewDocumentCommand \problemspace { o m }
{
  \group_begin:
    \IfValueT {#1} { \keys_set:nn { problib } { spacestyle = #1 } }
    \__problib_space:n {#2}
  \group_end:
}

```

exam defines `\part` inside the `parts` environment, so `problib` wraps it (and `\subpart`) right after the environment starts, but only within problems inserted by `\useproblem`. Each part inserts the space of the previous one; the end of the environment inserts the space of the last. Parts inside a `solution` are not counted.

```

\int_new:N \g__problib_part_int
\int_new:N \g__problib_subpart_int
\bool_new:N \g__problib_has_sub_bool

\cs_new:Npn \__problib_item:Nn #1#2
  { \clist_item:Nn #1 { \int_min:nn {#2} { \clist_count:N #1 } } }

\cs_new_protected:Npn \__problib_unless_solution:n #1
  { \legacy_if:nF { @insolution } {#1} }

\cs_new_protected:Npn \__problib_part_space:
{
  \bool_if:NF \g__problib_has_sub_bool
  {
    \__problib_space:e
    { \__problib_item:Nn \l__problib_partspace_clist \g__problib_part_int }
  }
}

\cs_new_protected:Npn \__problib_subpart_space:
{
  \__problib_space:e
  { \__problib_item:Nn \l__problib_subpartspace_clist \g__problib_subpart_int }
}

\cs_new_protected:Npn \__problib_next_part:
{
  \__problib_unless_solution:n
  {
    \int_compare:nNnT \g__problib_part_int > \c_zero_int { \__problib_part_space: }
    \int_gincr:N \g__problib_part_int
    \int_gzero:N \g__problib_subpart_int
    \bool_gset_false:N \g__problib_has_sub_bool
  }
}

```

```

    }
  }
\cs_new_protected:Npn \__problib_next_subpart:
{
  \__problib_unless_solution:n
  {
    \int_compare:nNnT \g__problib_subpart_int > \c_zero_int
    { \__problib_subpart_space: }
    \int_gincr:N \g__problib_subpart_int
  }
}

\cs_new_protected:Npn \__problib_wrap:NNN #1#2#3
{
  \cs_set_eq:cN { \__problib_ \cs_to_str:N #1 :w } #1
  \cs_set_eq:cN { \__problib_ \cs_to_str:N #2 :w } #2
  \cs_set_protected:Npe #1 { \exp_not:N #3 \exp_not:c { \__problib_ \cs_to_str:N #1 :w } }
  \cs_set_protected:Npe #2 { \exp_not:N #3 \exp_not:c { \__problib_ \cs_to_str:N #2 :w } }
}

\AddToHook { cmd/parts/after }
{
  \bool_if:NT \l__problib_active_bool
  { \__problib_wrap:NNN \part \bonuspart \__problib_next_part: }
}

\AddToHook { cmd/subparts/after }
{
  \bool_if:NT \l__problib_active_bool
  { \__problib_wrap:NNN \subpart \bonussubpart \__problib_next_subpart: }
}

\AddToHook { env/parts/end }
{
  \bool_if:NT \l__problib_active_bool
  {
    \__problib_unless_solution:n
    { \int_compare:nNnT \g__problib_part_int > \c_zero_int { \__problib_part_space: } }
  }
}

\AddToHook { env/subparts/end }
{
  \bool_if:NT \l__problib_active_bool
  {
    \__problib_unless_solution:n
    {
      \int_compare:nNnT \g__problib_subpart_int > \c_zero_int
      { \__problib_subpart_space: }
      \bool_gset_true:N \g__problib_has_sub_bool
    }
  }
}

```

8.7 Problems

`\useproblem` normalizes the path, records the use and inputs the file inside a group, so its options stay local to the problem. The `problem` environment stores the metadata, issues `\question` and adds the

question's space when it had no parts.

```

\prop_new:N \l__probib_meta_prop
\regex_const:Nn \c__probib_ext_regex { \A (\.\/)+ / \.tex \Z }

\cs_new_protected:Npn \__probib_meta:n #1
{ \prop_put:Nnn \l__probib_meta_prop {#1} { } }
\cs_new_protected:Npn \__probib_meta:nn #1#2
{ \prop_put:Nnn \l__probib_meta_prop {#1} {#2} }

\NewExpandableDocumentCommand \problemmeta { m }
{ \prop_item:Nn \l__probib_meta_prop {#1} }

\NewDocumentCommand \useproblem { O{ } m }
{
  \group_begin:
    \bool_set_true:N \l__probib_active_bool
    \keys_set:nn { probib / use } {#1}
    \str_set:Nn \l__probib_path_str {#2}
    \tl_trim_spaces:N \l__probib_path_str
    \regex_replace_all:NnN \c__probib_ext_regex { } \l__probib_path_str
    \file_if_exist:nTF { \l__probib_library_str \l__probib_path_str .tex }
    {
      \__probib_scan_usage:
      \bool_if:NF \g__probib_catalog_bool
      {
        \iow_now:Ne \g__probib_iow
        { \token_to_str:N \probibuse { \l__probib_path_str } }
        \__probib_check_reuse:
      }
      \file_input:n { \l__probib_library_str \l__probib_path_str .tex }
    }
    { \msg_error:nne { probib } { missing } { \l__probib_path_str } }
  \group_end:
}

\NewDocumentEnvironment { problem } { O{ } }
{
  \prop_clear:N \l__probib_meta_prop
  \keyval_parse:nnn \__probib_meta:n \__probib_meta:nn {#1}
  \int_gzero:N \g__probib_part_int
  \bool_if:NT \l__probib_newpage_bool { \newpage }
  \tl_if_empty:NT \l__probib_points_tl
  { \prop_get:NnNF \l__probib_meta_prop { points } \l__probib_points_tl { } }
  \tl_if_blank:VTF \l__probib_points_tl
  { \question }
  { \use:e { \exp_not:N \question [ \l__probib_points_tl ] } }
  \bool_if:NF \g__probib_catalog_bool
  { \__probib_catalog_note: }
  { \bool_if:NT \l__probib_showdata_bool { \__probib_data_note: } }
}
{
  \int_compare:nNnT \g__probib_part_int = \c_zero_int
  { \__probib_space:V \l__probib_problemspace_tl }
}

```

8.8 Notes and tables

The heading, metadata table and usage table shared by `showdata` notes and the catalog.

```
\tl_new:N \l__probib_rows_tl
\int_new:N \l__probib_cell_int
\bool_new:N \l__probib_show_metadata_bool
\bool_new:N \l__probib_show_usage_bool
\bool_new:N \l__probib_show_solution_bool

\cs_new_protected:Npn \__probib_meta_key: { \leavevmode \sffamily \color { gray } }

\cs_new_protected:Npn \__probib_heading:
{
  \texttt { \l__probib_path_str }
  \prop_get:NnNT \l__probib_meta_prop { title } \l__probib_rows_tl
  { \quad \textit { \mdseries \l__probib_rows_tl } }
  \par
}
```

Metadata other than the title, two key/value pairs per row.

```
\cs_new_protected:Npn \__probib_meta_table:
{
  \tl_clear:N \l__probib_rows_tl
  \int_zero:N \l__probib_cell_int
  \prop_map_inline:Nn \l__probib_meta_prop
  {
    \str_if_eq:nnF {##1} { title }
    {
      \int_incr:N \l__probib_cell_int
      \tl_put_right:Nn \l__probib_rows_tl { ##1 & ##2 }
      \tl_put_right:Ne \l__probib_rows_tl
      { \int_if_odd:nTF \l__probib_cell_int { & } { \exp_not:N \\\ } }
    }
  }
  \tl_if_empty:NF \l__probib_rows_tl
  {
    \begin{tabularx} { \linewidth }
    {
      @{} >{ \__probib_meta_key: } l >{ \raggedright \arraybackslash } X
      @{} \hspace { 1.5em } }
      >{ \__probib_meta_key: } l >{ \raggedright \arraybackslash } X @{}
    }
    \l__probib_rows_tl
  \end{tabularx}
  \par
}
}
```

Uses of the current problem, newest first, excluding the assignment being compiled.

```
\cs_new_protected:Npn \__probib_usage_table:
{
  \tl_clear:N \l__probib_rows_tl
  \cs_set_protected:Npn \probibentry ##1##2##3##4##5##6
  {
    \str_if_eq:eeF { \tl_to_str:n { ##2 | ##3 | ##4 } } { \g__probib_current_str }
    { \tl_put_right:Nn \l__probib_rows_tl { ##1 & ##3 & ##4 & ##5 & ##2 \\\ } }
  }
}
```

```

    }
    \l__proplib_uses_tl
    \tl_if_empty:NTF \l__proplib_rows_tl
    { { \__proplib_meta_key: Not-used-before. } \par }
    {
        \begin{tabularx} { \linewidth }
        { @{} l l l l >{ \raggedright \arraybackslash } X @{} }
        \__proplib_meta_key: date & \__proplib_meta_key: course &
        \__proplib_meta_key: term & \__proplib_meta_key: type &
        \__proplib_meta_key: assignment \\
        \l__proplib_rows_tl
        \end{tabularx}
    }
    \par
}

\cs_new_protected:Npn \__proplib_data_note:
{
    \leavevmode
    \group_begin:
    \footnotesize \color { gray }
    \__proplib_heading:
    \__proplib_usage_table:
    \group_end:
    \smallskip
}

\cs_new_protected:Npn \__proplib_catalog_note:
{
    { \bfseries \__proplib_heading: }
    \group_begin:
    \footnotesize
    \bool_if:NT \l__proplib_show_metadata_bool { \__proplib_meta_table: }
    \bool_lazy_and:nnT \l__proplib_show_metadata_bool \l__proplib_show_usage_bool
    { \smallskip }
    \bool_if:NT \l__proplib_show_usage_bool { \__proplib_usage_table: }
    \group_end:
    \smallskip
}

```

8.9 The catalog

`\proplibcatalog` passes its filters to Lua, which scans the library and the roots, writes the usage database and returns the matching paths. The problems are then inserted with `\useproblem`, without answer space.

```

\str_new:N \l__proplib_cat_folders_str
\str_new:N \l__proplib_cat_tags_str
\str_new:N \l__proplib_cat_topics_str
\str_new:N \l__proplib_cat_difficulty_str
\str_new:N \l__proplib_cat_unusedsince_str
\str_new:N \l__proplib_cat_sort_str
\clist_new:N \l__proplib_cat_clist

\cs_new_protected:Npn \__proplib_set_show:n #1
{

```

```

\clist_map_inline:nn { metadata, usage, solution }
{ \bool_set_false:c { l__proplib_show_ ##1 _bool } }
\clist_map_inline:nn {#1}
{
  \bool_if_exist:cTF { l__proplib_show_ ##1 _bool }
  { \bool_set_true:c { l__proplib_show_ ##1 _bool } }
  { \msg_error:nnn { proplib } { bad-show } {##1} }
}
}

\keys_define:nn { proplib / filter }
{
  folders      .str_set:N = \l__proplib_cat_folders_str ,
  tags         .str_set:N = \l__proplib_cat_tags_str ,
  topics       .str_set:N = \l__proplib_cat_topics_str ,
  difficulty   .str_set:N = \l__proplib_cat_difficulty_str ,
  unusedsince  .str_set:N = \l__proplib_cat_unusedsince_str ,
  sort         .choices:nn = { path, difficulty, lastused }
  { \str_set:NV \l__proplib_cat_sort_str \l_keys_choice_tl } ,
  sort         .initial:n = path ,
  show         .code:n     = \__proplib_set_show:n {#1} ,
  show         .initial:n = { metadata, usage } ,
}

\cs_new:Npn \__proplib_lua_opt:nN #1#2 { #1 = " \lua_escape:e {#2} " , }

\NewDocumentCommand \proplibcatalog { O{} }
{
  \bool_if:NTF \g__proplib_catalog_bool
  { \__proplib_catalog:n {#1} }
  { \msg_error:nn { proplib } { not-catalog } }
}

\cs_new_protected:Npn \__proplib_catalog:n #1
{
  \group_begin:
  \keys_set:nn { proplib / filter } {#1}
  \clist_set:Ne \l__proplib_cat_clist
  {
    \lua_now:e
    {
      proplib.catalog {
        \__proplib_lua_opt:nN { roots }          \l__proplib_roots_str
        \__proplib_lua_opt:nN { folders }        \l__proplib_cat_folders_str
        \__proplib_lua_opt:nN { tags }           \l__proplib_cat_tags_str
        \__proplib_lua_opt:nN { topics }         \l__proplib_cat_topics_str
        \__proplib_lua_opt:nN { difficulty }     \l__proplib_cat_difficulty_str
        \__proplib_lua_opt:nN { unusedsince }    \l__proplib_cat_unusedsince_str
        \__proplib_lua_opt:nN { sort }           \l__proplib_cat_sort_str
      }
    }
  }

  \bool_if:NF \g__proplib_db_bool { \__proplib_load_db:n { usage-db.tex } }
  \str_clear:N \l__proplib_library_str
  \tl_clear:N \l__proplib_problemspace_tl
  \clist_clear:N \l__proplib_partspace_clist

```

```

\clist_clear:N \l__probib_subpartspace_clist
\bool_if:NTF \l__probib_show_solution_bool
{ \legacy_if_set_true:n { printanswers } }
{ \legacy_if_set_false:n { printanswers } }
\clist_if_empty:NTF \l__probib_cat_clist
{ \emph { No-problems~match. } \par }
{
  \begin { questions }
    \clist_map_inline:Nn \l__probib_cat_clist { \useproblem {##1} }
  \end { questions }
}
\group_end:
}

```

8.10 Options and mode

In catalog mode the Lua code at the end of this file is loaded: `\file_input_stop:`, and Lua reads the text between the `--[[probib.lua]]` markers from the same file. Otherwise the record is opened at the start of the document.

```

\ProcessKeyOptions [ probib ]

\bool_if:NTF \g__probib_catalog_bool
{
  \sys_if_engine luatex:TF
  {
    \lua_now:n
    {
      local~f=io.open(kpse.find_file("probib.sty"));
      local~src=f:read("a");
      f:close();
      local~_,s=src:find("--[[probib"..".lua]]",1,true);
      local~t=s~and~src:find("--[[/probib"..".lua]]",s,true);
      assert(t,"probib:~Lua~code~not~found~in~probib.sty");
      probib=load(src:sub(s+1,t-1),"=probib.lua")();
    }
  }
  { \msg_error:nn { probib } { no-luatex } }
}
{
  \AtBeginDocument { \__probib_begin_assignment: }
  \AtEndDocument { \iow_close:N \g__probib_iow }
}

\file_input_stop:

```

8.11 Lua code

The library is every `.tex` file below the catalog's folder that contains a `problem` environment and no `\documentclass`. Metadata is read from the environment's optional argument. Hidden files and symbolic links are skipped.

```

--[[probib.lua]]

local probib = {}

local function read_file(path)

```

```

    local f = io.open(path, "r")
    if not f then return nil end
    local s = f:read("a")
    f:close()
    return s
end

local function warn(msg)
    texio.write_nl("term and log", "Package probleb Warning: " .. msg)
end

local function trim(s)
    s = s:match("^%s*(.)%s*$")
    if s:match("^%b{1}$") then s = s:sub(2, -2):match("^%s*(.)%s*$") end
    return s
end

-- Split on commas outside braces.
local function split(s)
    local items, depth, start = {}, 0, 1
    for i = 1, #s do
        local c = s:byte(i)
        if c == 123 then depth = depth + 1
        elseif c == 125 then depth = depth - 1
        elseif c == 44 and depth == 0 then
            items[#items + 1] = s:sub(start, i - 1)
            start = i + 1
        end
    end
    items[#items + 1] = s:sub(start)
    return items
end

local function set_of(s)
    local t = {}
    for _, item in ipairs(split(s or "")) do
        item = trim(item):lower()
        if item ~= "" then t[item] = true end
    end
    return t
end

local function walk(dir, ext, out, rel)
    local ok, iter, state = pcall(lfs.dir, dir)
    if not ok then return out end
    for name in iter, state do
        if name:sub(1, 1) ~= "." then
            local full = dir .. "/" .. name
            local mode = lfs.symlinkattributes(full, "mode")
            if mode == "directory" then
                walk(full, ext, out, rel .. name .. "/")
            elseif mode == "file" and name:sub(-#ext) == ext then
                out[#out + 1] = { full = full, rel = rel .. name:sub(1, -#ext - 1) }
            end
        end
    end
end

```

```

    return out
end

-- Returns the metadata table of a problem file, or nil if it is not one.
local function parse_problem(text)
    text = ("\\n" .. text):gsub("(\\[\\])%%[\\n]*", "%1")
    if text:find("\\documentclass") then return nil end
    local _, e = text:find("\\begin%*{problem}%s*")
    if not e then return nil end
    local meta = {}
    if text:sub(e + 1, e + 1) ~= "[" then return meta end
    local depth = 0
    for i = e + 2, #text do
        local c = text:byte(i)
        if c == 123 then depth = depth + 1
        elseif c == 125 then depth = depth - 1
        elseif c == 93 and depth == 0 then
            for _, item in ipairs(split(text:sub(e + 2, i - 1))) do
                local k, v = item:match("^%s*([~=-])%s*=(.*)$")
                if k and k ~= "" then meta[k] = trim(v) end
            end
            break
        end
    end
    return meta
end

```

Usage records are collected from every .pbu file below the roots. Identical uses (the same problem, date, name, course and term, as happens when an assignment folder is copied) are counted once.

```

local function collect_usage(roots)
    local usage, seen = {}, {}
    local home = os.getenv("HOME") or "~"
    for _, root in ipairs(split(roots)) do
        root = trim(root):gsub("^~", home):gsub("/$", "")
        if root ~= "" then
            if lfs.attributes(root, "mode") ~= "directory" then
                warn("usage root '" .. root .. "' not found")
            end
            for _, f in ipairs(walk(root, ".pbu", {}, "")) do
                local s = read_file(f.full)
                local rec = s and s:match("\\problibrecond%s*(%b{ })")
                if rec then
                    local r = {}
                    for k, v in rec:sub(2, -2):gmatch("(%w+)%s*=%s*(%b{ })") do r[k] = v:sub(2, -2) end
                    if r.record ~= "false" then
                        local entry = {
                            r.date or "", r.name or "", r.course or "", r.term or "", r.type or "", f.full
                        }
                        local id = table.concat(entry, "\\0", 1, 4)
                        for p in s:gmatch("\\proplibuse%s*(%b{ })") do
                            p = p:sub(2, -2)
                            local key = p .. "\\0" .. id
                            if not seen[key] then
                                seen[key] = true
                                usage[p] = usage[p] or {}
                                table.insert(usage[p], entry)
                            end
                        end
                    end
                end
            end
        end
    end
end

```

```

        end
    end
end
end
end
end
end
end
for _, uses in pairs(usage) do
    table.sort(uses, function(a, b)
        if a[1] ~= b[1] then return a[1] > b[1] end
        return a[2] < b[2]
    end)
end
return usage
end

local function write_db(usage, path)
    local paths = {}
    for p in pairs(usage) do paths[#paths + 1] = p end
    table.sort(paths)
    local lines = {}
    for i, p in ipairs(paths) do
        local entries = {}
        for j, u in ipairs(usage[p]) do
            entries[j] = "\\probibentry{" .. table.concat(u, "{") .. "}"
        end
        lines[i] = "\\probibusage{" .. p .. "}" .. table.concat(entries, "{") .. "}"
    end
    local f = io.open(path, "w")
    if not f then
        warn("cannot write '" .. path .. "'")
        return
    end
    f:write(table.concat(lines, "\n"), "\n")
    f:close()
end

local cache

local function scan(roots)
    if cache then return cache end
    local problems = {}
    for _, f in ipairs(walk(lfs.currentdir(), ".tex", {}, "")) do
        local text = read_file(f.full)
        local meta = text and parse_problem(text)
        if meta then problems[#problems + 1] = { path = f.rel, meta = meta } end
    end
    local usage = collect_usage(roots)
    write_db(usage, "usage-db.tex")
    cache = { problems = problems, usage = usage }
    return cache
end

```

Filtering and sorting. The result is returned to T_EX as a comma list of paths with string catcodes.

```

local function any(want, have)
    for w in pairs(want) do

```

```

    if have[w] then return true end
end
return false
end

function probleb.catalog(opts)
    local data = scan(opts.roots)
    local usage = data.usage

    local folders = {}
    for _, f in ipairs(split(opts.folders)) do
        f = trim(f):gsub("/$", "")
        if f ~= "" then folders[#folders + 1] = f end
    end
    local tags, topics = set_of(opts.tags), set_of(opts.topics)
    local lo, hi
    if opts.difficulty ~= "" then
        lo, hi = opts.difficulty:match("^s*(%d+)%s*%-?%s*(%d*)%s*$")
        if lo then
            lo, hi = tonumber(lo), tonumber(hi) or tonumber(lo)
        else
            warn("ignoring difficulty filter '" .. opts.difficulty .. "'")
        end
    end
    local since = trim(opts.unusedsince)

    local function last(p)
        local u = usage[p.path]
        return u and u[1][1] or ""
    end

    local function keep(p)
        if #folders > 0 then
            local hit = false
            for _, f in ipairs(folders) do
                if p.path == f or p.path:sub(1, #f + 1) == f .. "/" then hit = true break end
            end
            if not hit then return false end
        end
        if next(tags) and not any(tags, set_of(p.meta.tags)) then return false end
        if next(topics) and not any(topics, set_of(p.meta.topics)) then return false end
        if lo then
            local d = tonumber(p.meta.difficulty)
            if not d or d < lo or d > hi then return false end
        end
        if since ~= "" and last(p) >= since then return false end
        return true
    end

    local result = {}
    for _, p in ipairs(data.problems) do
        if keep(p) then result[#result + 1] = p end
    end

    local key
    if opts.sort == "difficulty" then

```

```

    key = function(p) return tonumber(p.meta.difficulty) or math.huge end
elseif opts.sort == "lastused" then
    key = last
end
table.sort(result, function(a, b)
    if key then
        local x, y = key(a), key(b)
        if x ~= y then return x < y end
    end
    return a.path < b.path
end)

local paths = {}
for i, p in ipairs(result) do paths[i] = p.path end
tex.sprint(-2, table.concat(paths, ","))
end

return probleb
--[[/probleb.lua]]
</package>

```